

Database Migration Management

Comprehensive guide to managing Supabase database migrations for SEMS2

Supabase Database Migration Management

Overview

Database migrations are version-controlled schema changes that enable systematic evolution of the database structure. Migrations ensure that schema modifications are applied consistently across development, staging, and production environments while maintaining data integrity and enabling rollback capabilities.

This guide covers the complete migration workflow for the SEMS2 Supabase deployment, including creation, testing, deployment, and troubleshooting procedures.

What are Database Migrations?

Database migrations are SQL scripts that define incremental changes to the database schema. Each migration file:

- Contains a timestamp prefix ensuring chronological execution order
- Defines a specific set of schema changes (table creation, column additions, index modifications)
- Is applied once and tracked by the database to prevent duplicate execution
- Can be rolled back if issues are detected

Benefits:

- **Version Control:** Schema changes are tracked in Git alongside application code
- **Reproducibility:** Consistent schema across all environments
- **Rollback Capability:** Revert problematic changes systematically
- **Team Collaboration:** Multiple developers can manage schema changes without conflicts
- **Audit Trail:** Complete history of database structure evolution

Prerequisites

Before managing migrations, ensure the following requirements are met:

Required Tools

- Supabase CLI: Verify installation with:

```
supabase --version
```



If not installed, refer to the [installation guide](#).

Production Credentials

The following credentials are required for production deployments:

Credential	Description	Location
POSTGRES_PASSWORD	PostgreSQL admin password	Dokploy Supabase service environment variables
POOLER_TENANT_ID	Database connection pooler tenant identifier	Supabase configuration
HOSTNAME	Production Supabase hostname	Dokploy domain configuration

Prototype Reference

In the SEMS2 prototype deployment:

- POOLER_TENANT_ID : mists-sems2
- HOSTNAME : sems2.app

These values are specific to the prototype environment. Production deployments should use organization-specific identifiers.

Local Development Environment

- Docker:** Required for local Supabase instance
- Project Access:** Cloned SEMS2 repository with write permissions
- Database Connection:** Stable network connection to production database

Migration Workflow

1. Create a New Migration

Generate a new migration file using the Supabase CLI:

```
supabase migration new <migration_name>
```

Naming Conventions:

- Use descriptive, lowercase names with underscores
- Prefix with action: `add_`, `create_`, `update_`, `remove_`
- Be specific about the change

Examples:

```
supabase migration new add_event_capacity_column
supabase migration new create_attendance_tracking_table
supabase migration new update_user_profile_indexes
```

The CLI generates a timestamped SQL file in `supabase/migrations/`:

```
supabase/migrations/20251025164806_add_event_capacity_column.sql
```

⚠ Important: Timestamp Preservation

Do not modify the timestamp prefix (e.g., `20251025164806`). This ensures migrations execute in chronological order. Only customize the descriptive name portion.

2. Author the Migration SQL

Edit the generated migration file and define the schema changes using SQL DDL statements.

Best Practices:

1. **Idempotency:** Write migrations that can run multiple times safely

```
-- Good: Idempotent column addition
```

```
ALTER TABLE events
ADD COLUMN IF NOT EXISTS capacity INTEGER;

-- Good: Idempotent table creation
CREATE TABLE IF NOT EXISTS attendance (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID REFERENCES events(id),
  user_id UUID REFERENCES profiles(id)
);
```

2. Atomicity: Group related changes in a single transaction

```
BEGIN;

ALTER TABLE events ADD COLUMN capacity INTEGER;
ALTER TABLE events ADD COLUMN registered_count INTEGER DEFAULT 0;

COMMIT;
```

3. Data Preservation: When modifying columns, migrate existing data

```
-- Add new column
ALTER TABLE profiles ADD COLUMN full_name TEXT;

-- Migrate existing data
UPDATE profiles
SET full_name = first_name || ' ' || last_name
WHERE full_name IS NULL;

-- Add constraint after data migration
ALTER TABLE profiles ALTER COLUMN full_name SET NOT NULL;
```

4. Index Creation: Create indexes for frequently queried columns

```
CREATE INDEX IF NOT EXISTS idx_events_start_date
ON events(start_date);
```

5. Comments: Document complex changes

```
-- Add capacity tracking for event registration limits
-- Supports waitlist functionality (Issue #123)
ALTER TABLE events ADD COLUMN capacity INTEGER;
```

Common Migration Patterns:

- **Add Column:** `ALTER TABLE table_name ADD COLUMN column_name TYPE;`
- **Create Table:** `CREATE TABLE IF NOT EXISTS table_name (...);`
- **Add Foreign Key:** `ALTER TABLE table_name ADD CONSTRAINT fk_name FOREIGN KEY (column) REFERENCES other_table(id);`
- **Create Index:** `CREATE INDEX idx_name ON table_name(column);`
- **Add RLS Policy:** `CREATE POLICY policy_name ON table_name FOR SELECT USING (...);`

3. Test Migrations Locally

Before deploying to production, thoroughly test migrations in the local development environment.

Apply Pending Migrations

Execute all pending migrations against the local database:

```
supabase migration up
```

This command:

- Identifies unapplied migrations by comparing timestamps
- Executes SQL files in chronological order
- Updates the migration tracking table
- Reports success or failure for each migration

Reset Database (Clean Slate)

For comprehensive testing, reset the database to a clean state:

```
supabase db reset
```

This command:

- Drops all database objects (tables, functions, triggers)
- Recreates the database from scratch
- Applies all migrations in order
- Executes seed data scripts (if configured)

⚠ Warning: Destructive Operation

`supabase db reset` permanently deletes all local database data. Use only in development environments. Never execute against production databases.

Verification Steps

After applying migrations, verify the changes:

1. **Schema Verification:** Confirm tables, columns, and constraints exist

```
-- Check table structure
\d table_name

-- List all tables
\dt

-- View indexes
\di
```



2. **Data Integrity:** Verify existing data remains intact

```
SELECT COUNT(*) FROM critical_table;
```



3. **Application Testing:** Run the application against the updated schema

- Execute automated test suites
- Perform manual testing of affected features
- Verify API endpoints function correctly

4. Deploy to Production

After successful local testing, deploy migrations to the production database.

Configure SSL Mode

The Supabase CLI has a known issue with SSL connections to self-hosted instances. Configure the environment to disable SSL verification:

Windows (PowerShell):

```
$env:PGSSLMODE = "disable"
```



macOS/Linux (Bash):

```
export PGSSLMODE=disable
```



Known Issue: SSL Connection Errors

Self-hosted Supabase instances may encounter SSL certificate validation errors with the Supabase CLI. The workaround disables SSL mode for the connection.

Issue Tracker:[supabase/cli#4142](https://github.com/supabase/cli/issues/4142)

Verify this issue status before deploying, as it may be resolved in newer CLI versions.

Execute Migration Deployment

Deploy migrations using the `db push` command with the production connection string:

Command Structure:

```
supabase db push --db-url "postgres://postgres.<POOLER_TENANT_ID>:<POSTGRES_PASSWORD>@<HOSTNAME>:5432/postgres"
```



Parameter Definitions:

- `postgres.<POOLER_TENANT_ID>`: Database user with pooler tenant identifier
- `<POSTGRES_PASSWORD>`: PostgreSQL admin password
- `<HOSTNAME>`: Production Supabase domain
- `5432`: PostgreSQL default port
- `postgres`: Default database name

Prototype Example

Example deployment command from the SEMS2 prototype:

```
supabase db push --db-url "postgres://postgres.mists-sems2:n0TaRea1Passw0rd@sems2.app:5432/postgres"
```

Note: This example uses prototype credentials for demonstration. Production deployments must use secure, organization-specific credentials.

Security Best Practices:

1. Environment Variables: Store credentials in environment variables

```
# Windows PowerShell
$env:DB_PASSWORD = "secure_password"
supabase db push --db-url "postgres://postgres.tenant:$env:DB_PASSWORD@host:5432/postgres"
```

```
# macOS/Linux Bash
export DB_PASSWORD="secure_password"
supabase db push --db-url "postgres://postgres.tenant:$DB_PASSWORD@host:5432/postgres"
```

2. Secrets Management: Use a secrets manager for production credentials

- Azure Key Vault
- AWS Secrets Manager
- HashiCorp Vault
- 1Password CLI

3. Access Control: Restrict production database access to authorized personnel only

Post-Deployment Verification

After deployment, verify the migration was applied successfully:

1. Check Migration Status: Access Supabase Studio → SQL Editor and execute:

```
SELECT * FROM supabase_migrations.schema_migrations
ORDER BY version DESC
LIMIT 5;
```

2. Verify Schema Changes: Confirm the expected tables, columns, and indexes exist:

```
-- Example: Verify new column
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'events' AND column_name = 'capacity';
```

3. Test Application Functionality:

- Execute critical user workflows

- Verify affected features function correctly
- Monitor application logs for database errors

4. Performance Monitoring:

- Check query performance for modified tables
- Verify indexes are being utilized
- Monitor database resource utilization

Rollback Procedures

If a migration causes issues in production, follow these rollback procedures:

Option 1: Write a Reverse Migration

Create a new migration that reverses the problematic changes:

```
supabase migration new rollback_add_event_capacity
```



Author SQL that undoes the previous migration:

```
-- Rollback for: 20251025164806_add_event_capacity_column.sql
ALTER TABLE events DROP COLUMN IF EXISTS capacity;
```



Deploy the rollback migration following the standard workflow.

Option 2: Database Restore

For critical failures, restore from a database backup:

1. **Identify Backup Point:** Locate the most recent backup before the migration
2. **Schedule Downtime:** Notify users of temporary service interruption
3. **Restore Database:** Follow Dokploy/Supabase restore procedures
4. **Verify Integrity:** Confirm data consistency after restore
5. **Update Migration Tracking:** Manually update the migration tracking table



Critical Warning

Database restoration results in data loss for any changes made after the backup timestamp. This should only be used as a last resort for catastrophic failures. Always attempt reverse migrations first.

Troubleshooting

SSL/Certificate Errors

Symptom: Connection fails with SSL certificate validation errors

Solution:

```
# Windows
$env:PGSSLMODE = "disable"

# macOS/Linux
export PGSSLMODE=disable
```



Root Cause: Self-hosted Supabase instances may use self-signed certificates that the CLI cannot validate.

Migration Ordering Issues

Symptom: Migrations execute in incorrect order, causing dependency failures

Solution:

1. Verify migration files have correct timestamp prefixes
2. Ensure no manual timestamp modifications
3. Reset local database and reapply:

```
supabase db reset
```



Prevention: Never modify migration file timestamps after creation.

Schema Drift Detection

Symptom: `supabase db push` rejects migration due to schema differences between local and production

Cause: Manual changes were made directly to the production database without corresponding migration files.

Solution:

1. **Capture Remote Changes:**

```
supabase db pull
```

This generates a migration file with the production schema state.

2. **Resolve Conflicts:**

- Review the generated migration
- Merge with local changes
- Test thoroughly in development

3. **Redeploy:**

```
supabase db push --db-url "<production_url>"
```

Prevention:

- Never make manual schema changes in production
- Always use migrations for all schema modifications
- Implement change control procedures

Foreign Key Constraint Violations

Symptom: Migration fails with foreign key constraint errors

Cause: Attempting to add foreign key constraints to columns with existing invalid references.

Solution:

```
-- First, clean invalid data
DELETE FROM child_table
WHERE parent_id NOT IN (SELECT id FROM parent_table);

-- Then add constraint
ALTER TABLE child_table
```

```
ADD CONSTRAINT fk_parent
FOREIGN KEY (parent_id) REFERENCES parent_table(id);
```

Permission Denied Errors

Symptom: `permission denied for schema public` or similar errors

Cause: Database user lacks required privileges.

Solution:

1. Verify using the `postgres` superuser account
2. Check database user permissions:

```
SELECT * FROM pg_roles WHERE rolname = 'postgres';
```



3. Grant necessary permissions if needed

Connection Timeout

Symptom: CLI hangs or times out during `db push`

Solutions:

- Verify network connectivity to production server
- Check firewall rules allow PostgreSQL port 5432
- Confirm database service is running in Dokploy
- Verify connection credentials are correct

Best Practices

Migration Development

1. **One Purpose Per Migration:** Each migration should address a single logical change
2. **Test Thoroughly:** Always test in development before production deployment
3. **Backup First:** Ensure recent database backups exist before deploying migrations
4. **Off-Peak Deployment:** Deploy schema changes during low-traffic periods

5. **Monitoring:** Watch application logs and database metrics after deployment

Code Review

- Have migrations reviewed by another team member
- Verify SQL syntax and logic
- Confirm backward compatibility with deployed application code
- Check for performance implications (table locks, index builds)

Documentation

- Comment complex migrations with explanations
- Update data models and architecture diagrams
- Document any manual steps required (data migrations, configuration changes)
- Link migrations to issue tracker tickets

Version Control

- Commit migration files with descriptive commit messages
- Tag releases with corresponding database schema versions
- Never modify committed migration files (create new migrations instead)
- Keep migration history intact (do not squash or rebase)

Additional Resources

- [Supabase CLI Documentation](#)
- [PostgreSQL DDL Reference](#)
- [Supabase Row Level Security](#)
- [Database Migration Best Practices](#)

For deployment-specific information, refer to the [SEMS2 Deployment Manual](#).

